

Feb 6th HW 15

1. Explain the fundamental differences between containers and virtual machines. What makes containers more lightweight and faster to start?
 - Containers and virtual machines differ mainly in how they virtualize computing resources. Virtual machines virtualize hardware by running a full guest operating system on top of a hypervisor, whereas containers virtualize the operating system by sharing the host OS kernel while isolating applications in user space. Because containers do not require a separate guest OS, they consume fewer resources such as CPU, memory, and disk. Containers start in seconds or milliseconds, while virtual machines typically take minutes to boot due to OS initialization. Virtual machines are commonly used when strong isolation or different operating systems are required, while containers are preferred for microservices, CI/CD pipelines, and cloud-native applications due to their lightweight nature and fast startup time.
2. Describe the role of Namespaces and Cgroups in containerization. Provide at least three examples of different namespace types and explain what each one isolates
 - Linux containers rely heavily on **Namespaces** and **Control Groups (cgroups)** for isolation and resource management. Namespaces provide isolation by making a set of system resources appear separate to processes. For example, the **PID namespace** isolates process IDs so containers have their own process tree, the **NET namespace** isolates network interfaces and IP addresses, and the **MNT namespace** isolates filesystem mount points. Cgroups control and limit resource usage such as CPU, memory, and I/O, preventing one container from exhausting host resources. Together, namespaces isolate *what a container can see*, while cgroups control *how much it can use*, forming the foundation of containerization.
3. Explain how container image layers work. What is the copy-on-write mechanism, and why is it important for container efficiency?
 - Container images are built using multiple read-only layers stacked on top of each other. Each layer represents a filesystem change, such as installing packages or copying application files. When a container starts, a thin writable layer is added on top of these read-only layers. The copy-on-write (CoW) mechanism ensures that when a file is modified, it is copied from the read-only layer into the writable layer instead of altering the original image. This design improves storage efficiency, speeds up image distribution, and allows multiple containers to share the same base image layers without duplication.
4. What is the Open Container Initiative (OCI)? Describe the three main OCI specifications and explain why standardization is important for the container ecosystem
 - The Open Container Initiative (OCI) is an open governance structure created to define standards for container formats and runtimes. OCI provides three main specifications: the Runtime Specification, which defines how a container should be executed; the Image Specification, which defines how container images are

built and structured; and the Distribution Specification, which defines how images are stored and distributed via registries. Standardization ensures interoperability across container tools and platforms, allowing containers built with one tool to run on another without modification

5. Describe the container runtime hierarchy from Kubernetes down to the Linux kernel. Explain the responsibilities of each layer (orchestration, high-level runtime, low-level runtime).
 - Container runtime architecture consists of multiple layers. At the top, Kubernetes acts as the orchestration layer, handling scheduling, scaling, and lifecycle management of containers. Beneath Kubernetes is the high-level runtime such as containerd or CRI-O, which manages container images, networking, and lifecycle operations. At the lowest level is the low-level runtime like runc, which directly interfaces with the Linux kernel to create containers using namespaces and cgroups. These layers interact in a hierarchical manner, where Kubernetes issues commands through CRI to the runtime, which then invokes runc to start containers
6. What is the Container Runtime Interface (CRI)? Explain how it enables Kubernetes to work with different container runtimes. What are the two main services it provides?
 - The Container Runtime Interface (CRI) is an API that allows Kubernetes to communicate with different container runtimes in a standardized way. CRI abstracts runtime implementation details, enabling Kubernetes to support multiple runtimes without code changes. It provides two main services: RuntimeService, which manages container lifecycle operations such as start, stop, and delete, and ImageService, which handles image pull, removal, and listing. This abstraction increases flexibility and future-proofs Kubernetes against runtime changes
7. Explain the historical problem Java had with containers before JDK 8u191/JDK 10. How did modern JDK versions solve this problem? What JVM options should be used for memory configuration in containers?
 - Before JDK 8u191 and JDK 10, the JVM was unaware of container resource limits and used host-level CPU and memory values, often leading to excessive memory usage and container crashes. Modern JDK versions detect cgroup limits and adjust heap sizing accordingly. JVM options such as `-XX:MaxRAMPercentage` and `-XX:InitialRAMPercentage` are now recommended instead of fixed heap sizes. This improvement is critical in production environments because it prevents out-of-memory errors and ensures predictable behavior in containerized deployments
8. Explain what multi-stage builds are and why they are important. Provide a practical example scenario where multi-stage builds would significantly reduce image size.
 - Multi-stage Docker builds allow multiple FROM statements in a single Dockerfile, enabling separation of build and runtime environments. For example,

a Java application can be compiled using a Maven image in one stage and then copied into a minimal JRE image in the final stage. This approach significantly reduces image size, removes unnecessary build tools, and improves security. Multi-stage builds are especially beneficial for Java and Go applications, where build dependencies are large but not needed at runtime

9. Describe the different Docker network modes (bridge, host, none, overlay). When would you use each mode? What is the role of CNI plugins in Kubernetes networking?
 - Docker provides several networking modes. Bridge mode is the default and allows containers to communicate through a virtual bridge. Host mode removes network isolation and lets containers use the host network directly, improving performance but reducing isolation. None mode disables networking entirely, useful for batch jobs. Overlay mode enables multi-host networking, commonly used in Docker Swarm and Kubernetes. In Kubernetes, CNI plugins such as Calico, Flannel, and Weave implement networking, handling pod-to-pod communication and network policies
10. Explain the different types of storage available in containers (volumes, bind mounts, tmpfs). What are the performance and persistence characteristics of each? When would you use each type?
 - Containers support multiple storage types. Named volumes are managed by Docker and provide persistent storage independent of container lifecycle. Bind mounts map host directories directly into containers, offering high performance but tight coupling to the host filesystem. tmpfs stores data in memory only and is extremely fast but non-persistent. Volumes are ideal for databases, bind mounts are useful during development, and tmpfs is best for sensitive or temporary data such as secrets